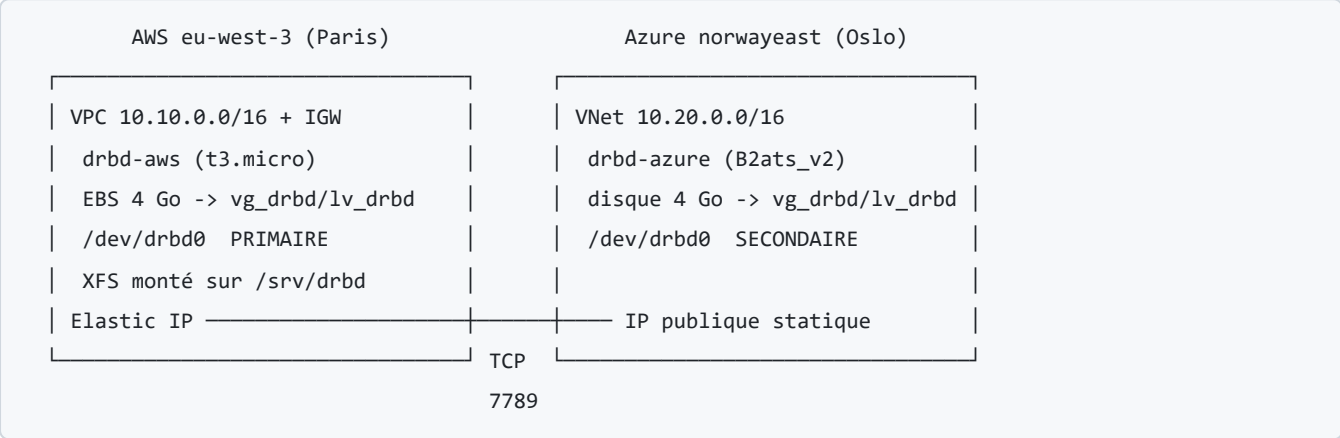
Exercice 2 — Réplication DRBD d'un LV entre AWS et Azure

Un logical volume LVM est répliqué de façon synchrone (DRBD 9, protocole C) entre une VM Rocky Linux 9 chez AWS et une VM Rocky Linux 9 chez Azure. Un seul `terraform apply` crée les deux côtés : un seul root module et deux providers, pour que chaque VM connaisse l'IP publique de son pair.

Architecture



- **Pile de stockage** (sur chaque nœud) : disque de données, PV, VG `vg_drbd`, LV `lv_drbd` (1 Go), `/dev/drbd0`, XFS (sur le primaire seulement).
- **Réseau** : les deux clouds font un NAT 1:1. Chaque VM ne voit que son IP privée. Le fichier `/etc/drbd.d/r0.res` diffère donc d'un nœud à l'autre : la section locale porte l'IP privée (écoute), celle du pair son IP publique (connexion).
- **Filtrage** : le port 7789 et l'ICMP sont ouverts uniquement à l'IP `/32` du pair. Le SSH est ouvert uniquement à `admin_cidr`, l'IP publique de l'admin détectée automatiquement par défaut.
- **Démarrage** : `templates/drbd-node.sh.tftpl`, exécuté par cloud-init. Il crée le LVM, installe DRBD depuis ELRepo (`kmod-drbd9x`, `drbd9x-utils`), écrit la ressource, puis lance `create-md` et `up`. Le primaire est promu et formaté.

Choix pour limiter les coûts

Poste	Choix	Pourquoi
VM AWS	`t3.micro`, `cpu_credits = "standard"`	éligible offre gratuite ; pas de facturation CPU « unlimited »
VM Azure	`Standard_B2ats_v2`	taille B la moins chère autorisée par l'abonnement Student
Région Azure	`norwayeast`	la policy Student n'autorise que 5 régions ; la série B n'est pas ouverte en France Central
Réseau AWS	VPC + IGW, **pas de NAT Gateway**	une NAT Gateway coûte environ 35 $/mois

Poste	Choix	Pourquoi
Liaison	IP publiques + filtrage `/32`, **pas de VPN Gateway**	une VPN Gateway Azure coûte plus de 25 $/mois
Disques	4 Go HDD/gp3, LV de 1 Go	synchro initiale courte et peu de trafic sortant

Après chaque séance, lancer `terraform destroy`. Arrêter les VM ne suffit pas : les disques et les IP publiques restent facturés.

Prérequis (une seule fois)

```
aws sts get-caller-identity   # utilisateur IAM avec AmazonEC2FullAccess
az account show               # abonnement Azure actif
az vm image terms accept --publisher resf --offer rockylinux-x86_64 --plan 9-base
cp terraform.tfvars.example terraform.tfvars   # renseigner azure_subscription_id
```

Déploiement

```
terraform init
terraform plan -out=tfplan
terraform apply tfplan
```

Le bootstrap prend 3 à 5 minutes après l'`apply` (installation de DRBD). Pour le suivre :

```
$(terraform output -raw ssh_aws) 'sudo tail -f /var/log/drbd-bootstrap.log'
```

Vérification

```
# État de la réplication (sur l'un ou l'autre nœud)
sudo drbdadm status r0
#  r0 role:Primary
#    disk:UpToDate
#    drbd-azure role:Secondary
#      peer-disk:UpToDate

cat /srv/drbd/hello.txt      # fichier écrit par le primaire au démarrage
```

Test de bascule (AWS vers Azure)

```
# sur drbd-aws
sudo umount /srv/drbd
sudo drbdadm secondary r0

# sur drbd-azure
sudo drbdadm primary r0
sudo mkdir -p /srv/drbd && sudo mount /dev/drbd0 /srv/drbd
cat /srv/drbd/hello.txt      # même contenu, répliqué depuis AWS
```

Validation réelle (03/11/2026)

Deux déploiements complets ont été faits.

1. **Premier** `apply` : il a révélé deux problèmes dans le script de démarrage. - `GPG check FAILED` : ELRepo signe avec sa clé v2, que le paquet `elrepo-release` de Rocky ne référence pas. Le template importe la clé explicitement. - `dnf` bloqué côté Azure : le `mirrorlist` d'ELRepo renvoyait vers un miroir injoignable depuis `norwayeast`. Le template passe par `baseurl`.
2. **Second** `apply`, avec le template corrigé : tout s'enchaîne sans intervention manuelle. Les captures ci-dessous viennent de ce déploiement.

Le bootstrap AWS dure environ 12 minutes, contre 3 côté Azure. `cpu_credits = "standard"` évite toute facturation CPU, mais bride le `t3.micro` à 10 % par vCPU. La reconstruction de l'initramfs par `dracut`, à l'installation du module DRBD, est donc lente.

Preuves

1. Déploiement Terraform

`terraform apply` : 24 ressources créées, IP publiques des deux nœuds en sortie.

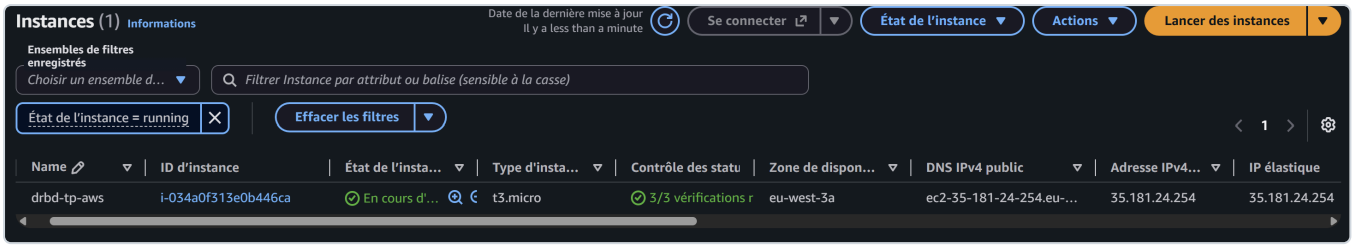
```
Apply complete! Resources: 24 added, 0 changed, 0 destroyed.

Outputs:

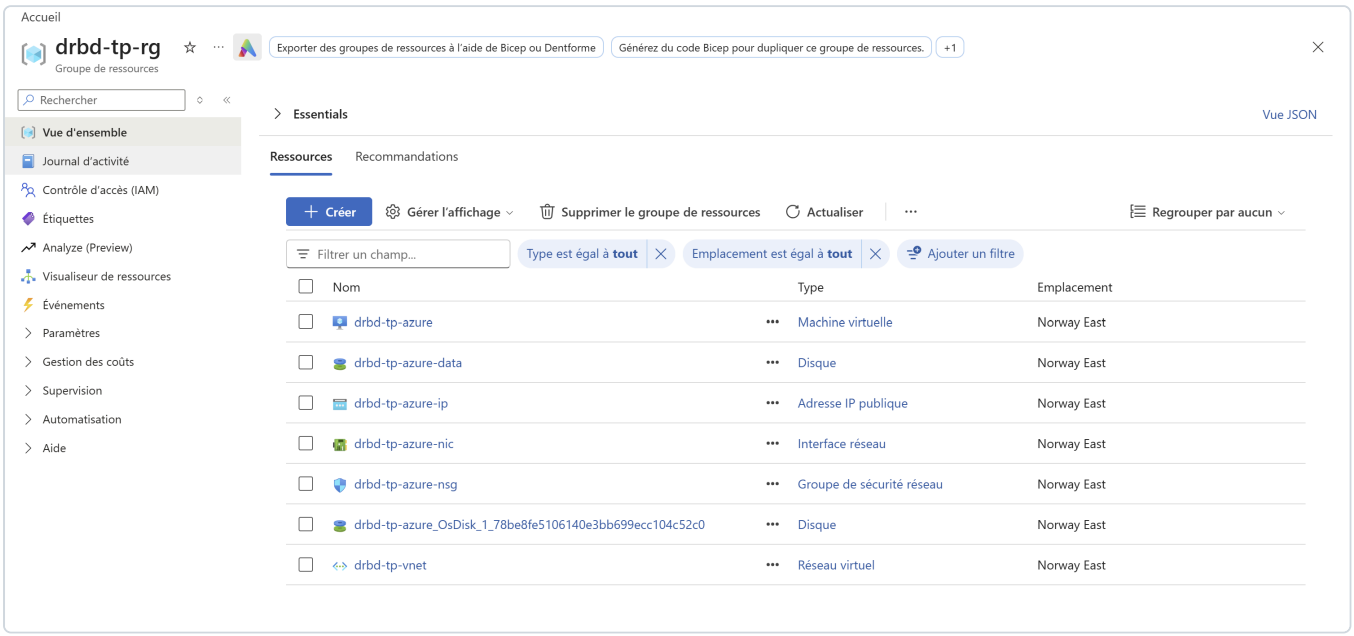
admin_cidr = "46.193.57.103/32"
aws_ami = "Rocky-9-EC2-Base-9.8-20260525.0.x86_64"
aws_public_ip = "35.181.24.254"
azure_public_ip = "20.251.28.85"
ssh_aws = "ssh -i ~/.ssh/tp_vm rocky@35.181.24.254"
ssh_azure = "ssh -i ~/.ssh/tp_vm rocky@20.251.28.85"

nolan in 🌐 debian in terraform-tp/tp2/exo2-drbd on  main [?] via ◆ default on ☁️  (eu-west-3) took 2m7s
🪛 0.27 ❯
```

Instance AWS `drbd-tp-aws` (`t3.micro`, `eu-west-3a`) en cours d'exécution, avec son Elastic IP :



Groupe de ressources Azure `drbd-tp-rg` (Norway East) : VM, disque de données, IP publique, NIC, NSG, disque OS, VNet :



2. Bootstrap (cloud-init), journal `/var/log/drbd-bootstrap.log` côté AWS

Création de la pile LVM sur le disque de données (`/dev/nvme1n1` , PV, `vg_drbd` , `lv_drbd`) :

```
 Userid     : "Rocky Enterprise Software Foundation - Release key 2022 <releng@rockylinux.org>"
 Fingerprint: 21CB 256A E16F C54C 6E65 2949 702D 426D 350D 275D
 From       : /etc/pki/rpm-gpg/RPM-GPG-KEY-Rocky-9
Key imported successfully
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Installing       : device-mapper-event-libs-9:1.02.207-4.el9.1.x86_64     1/7
  Installing       : libnvme-1.16.1-3.el9.x86_64                            2/7
  Installing       : libaio-0.3.111-13.el9.x86_64                           3/7
  Installing       : device-mapper-event-9:1.02.207-4.el9.1.x86_64          4/7
  Running scriptlet: device-mapper-event-9:1.02.207-4.el9.1.x86_64          4/7
Created symlink /etc/systemd/system/sockets.target.wants/dm-event.socket → /usr/lib/systemd/system/dm-event.socket.

  Installing       : lvm2-libs-9:2.03.33-4.el9.1.x86_64                     5/7
  Installing       : device-mapper-persistent-data-1.1.0-1.el9.x86_64       6/7
  Installing       : lvm2-9:2.03.33-4.el9.1.x86_64                          7/7
  Running scriptlet: lvm2-9:2.03.33-4.el9.1.x86_64                          7/7
Created symlink /etc/systemd/system/sysinit.target.wants/lvm2-monitor.service → /usr/lib/systemd/system/lvm2-monitor.servic
Created symlink /etc/systemd/system/sysinit.target.wants/lvm2-lvmpolld.socket → /usr/lib/systemd/system/lvm2-lvmpolld.socke

  Verifying        : device-mapper-event-9:1.02.207-4.el9.1.x86_64          1/7
  Verifying        : device-mapper-event-libs-9:1.02.207-4.el9.1.x86_64     2/7
  Verifying        : device-mapper-persistent-data-1.1.0-1.el9.x86_64       3/7
  Verifying        : libaio-0.3.111-13.el9.x86_64                           4/7
  Verifying        : libnvme-1.16.1-3.el9.x86_64                            5/7
  Verifying        : lvm2-9:2.03.33-4.el9.1.x86_64                          6/7
  Verifying        : lvm2-libs-9:2.03.33-4.el9.1.x86_64                     7/7

Installed:
  device-mapper-event-9:1.02.207-4.el9.1.x86_64
  device-mapper-event-libs-9:1.02.207-4.el9.1.x86_64
  device-mapper-persistent-data-1.1.0-1.el9.x86_64
  libaio-0.3.111-13.el9.x86_64
  libnvme-1.16.1-3.el9.x86_64
  lvm2-9:2.03.33-4.el9.1.x86_64
  lvm2-libs-9:2.03.33-4.el9.1.x86_64

Complete!
  Physical volume "/dev/nvme1n1" successfully created.
  Creating devices file /etc/lvm/devices/system.devices
  Volume group "vg_drbd" successfully created
  Logical volume "lv_drbd" created.
Last metadata expiration check: 0:00:49 ago on Tue 03 Nov 2026 10:03:55 AM UTC.
Dependencies resolved.
================================================================================
 Package               Architecture   Version             Repository      Size
================================================================================
Installing:
 elrepo-release        noarch         9.5-1.el9           extras          18 k
```

Installation de DRBD 9 depuis ELRepo (`kmod-drbd9x` , `drbd9x-utils`) :

```
Transaction Summary
================================================================================
Install  1 Package

Total download size: 18 k
Installed size: 8.1 k
Downloading Packages:
elrepo-release-9.5-1.el9.noarch.rpm             217 kB/s |  18 kB     00:00
--------------------------------------------------------------------------------
Total                                            32 kB/s |  18 kB     00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Installing       : elrepo-release-9.5-1.el9.noarch                        1/1
  Verifying        : elrepo-release-9.5-1.el9.noarch                        1/1

Installed:
  elrepo-release-9.5-1.el9.noarch

Complete!
ELRepo.org Community Enterprise Linux Repositor 167 kB/s | 240 kB     00:01
Last metadata expiration check: 0:00:01 ago on Tue 03 Nov 2026 10:05:00 AM UTC.
Dependencies resolved.
================================================================================
 Package            Architecture Version                    Repository    Size
================================================================================
Installing:
 drbd9x-utils       x86_64       9.35.0-1.el9.elrepo        elrepo       1.0 M
 kmod-drbd9x        x86_64       9.3.4-1.el9_8.elrepo       elrepo       501 k

Transaction Summary
================================================================================
Install  2 Packages

Total download size: 1.5 M
Installed size: 5.9 M
Downloading Packages:
(1/2): kmod-drbd9x-9.3.4-1.el9_8.elrepo.x86_64. 417 kB/s | 501 kB     00:01
(2/2): drbd9x-utils-9.35.0-1.el9.elrepo.x86_64. 734 kB/s | 1.0 MB     00:01
--------------------------------------------------------------------------------
Total                                           1.1 MB/s | 1.5 MB     00:01
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Installing       : kmod-drbd9x-9.3.4-1.el9_8.elrepo.x86_64                1/2
  Running scriptlet: kmod-drbd9x-9.3.4-1.el9_8.elrepo.x86_64                1/2
  Installing       : drbd9x-utils-9.35.0-1.el9.elrepo.x86_64                2/2
  Running scriptlet: drbd9x-utils-9.35.0-1.el9.elrepo.x86_64                2/2
  Running scriptlet: kmod-drbd9x-9.3.4-1.el9_8.elrepo.x86_64                2/2
```

Métadonnées DRBD, XFS sur `/dev/drbd0`, promotion en primaire. La synchro initiale vers Azure démarre (`SyncSource ... done:0.15`) :

```
  --==  Thank you for participating in the global usage survey  ==--
The server's response is:

you are the 14007th user to install this version

Do you want to proceed?
*** confirmation forced via --force option ***
initializing bitmap (32 KB) to all zero
md_offset 1073737728
al_offset 1073704960
bm_offset 1073672192

Found some data

 ==> This might destroy existing data! <==
initializing activity log
Writing meta data...
New drbd meta data block successfully created.
success
Created symlink /etc/systemd/system/multi-user.target.wants/drbd.service → /usr/lib/systemd/system/drbd.service.
meta-data=/dev/drbd0             isize=512    agcount=4, agsize=65532 blks
         =                       sectsz=4096  attr=2, projid32bit=1
         =                       crc=1        finobt=1, sparse=1, rmapbt=0
         =                       reflink=1    bigtime=1 inobtcount=1 nrext64=0
data     =                       bsize=4096   blocks=262127, imaxpct=25
         =                       sunit=0      swidth=0 blks
naming   =version 2              bsize=4096   ascii-ci=0, ftype=1
log      =internal log           bsize=4096   blocks=16384, version=2
         =                       sectsz=4096  sunit=1 blks, lazy-count=1
realtime =none                   extsz=4096   blocks=0, rtextents=0
r0 role:Primary
  disk:UpToDate open:yes
  drbd-azure role:Secondary
    replication:SyncSource peer-disk:Inconsistent done:0.15

=== 2026-11-03T10:12:38+00:00 bootstrap terminé
```

3. Réplication établie

Les deux nœuds sont `UpToDate` : AWS primaire, Azure secondaire.

```
nolan in 🌐 debian in terraform-tp/tp2/exo2-drbd on  main [?] via 🔹 default on ☁️  (eu-west-3) took 31m37s
🪛0.02 💀 $(terraform output -raw ssh_aws)
[rocky@drbd-aws ~]$ sudo drbdadm status r0
r0 role:Primary
  disk:UpToDate open:yes
  drbd-azure role:Secondary
    peer-disk:UpToDate

[rocky@drbd-aws ~]$
```

```
nolan in 🌐 debian in terraform-tp/tp2/exo2-drbd on  main [?] via 🔹 default on ☁️  (eu-west-3)
🪛0.19 ❯ $(terraform output -raw ssh_azure)
[rocky@drbd-azure ~]$ sudo drbdadm status r0
r0 role:Secondary
  disk:UpToDate open:no
  drbd-aws role:Primary
    peer-disk:UpToDate

[rocky@drbd-azure ~]$
```

4. Pile de stockage identique sur les deux clouds

Disque de 4 Go, `vg_drbd-lv_drbd` (1 Go), `drbd0`. Sur AWS, `drbd0` est monté sur `/srv/drbd` (primaire) ; sur Azure, il n'est pas monté (secondaire). Le nom du disque diffère (`nvme1n1` / `sdb`) : le script le repère par sa taille.

```
[rocky@drbd-aws ~]$ lsblk && sudo lvs
NAME              MAJ:MIN RM    SIZE RO TYPE MOUNTPOINTS
nvme0n1           259:0    0     10G  0 disk
├─nvme0n1p1       259:2    0      2M  0 part
├─nvme0n1p2       259:3    0    100M  0 part /boot/efi
├─nvme0n1p3       259:4    0   1000M  0 part /boot
└─nvme0n1p4       259:5    0    8.9G  0 part /
nvme1n1           259:1    0      4G  0 disk
└─vg_drbd-lv_drbd 253:0    0      1G  0 lvm
  └─drbd0         147:0    0 1023.9M  0 disk /srv/drbd
  LV      VG      Attr       LSize Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  lv_drbd vg_drbd -wi-ao---- 1.00g
[rocky@drbd-aws ~]$
```

```
[rocky@drbd-azure ~]$ lsblk && sudo lvs
NAME              MAJ:MIN RM    SIZE RO TYPE MOUNTPOINTS
sda                 8:0    0     10G  0 disk
├─sda1              8:1    0      2M  0 part
├─sda2              8:2    0    100M  0 part /boot/efi
├─sda3              8:3    0   1000M  0 part /boot
└─sda4              8:4    0    8.9G  0 part /
sdb                 8:16   0      4G  0 disk
└─vg_drbd-lv_drbd 253:0    0      1G  0 lvm
  └─drbd0         147:0    0 1023.9M  0 disk
sr0                11:0    1    642K  0 rom
  LV      VG      Attr       LSize Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  lv_drbd vg_drbd -wi-ao---- 1.00g
[rocky@drbd-azure ~]$
```

5. Bascule AWS vers Azure

Sur AWS : écriture dans `hello.txt`, démontage, passage en secondaire.

```
[rocky@drbd-aws ~]$ echo "test bascule $(date)" | sudo tee -a /srv/drbd/hello.txt
cat /srv/drbd/hello.txt
sudo umount /srv/drbd && sudo drbdadm secondary r0 && sudo drbdadm status r0
test bascule Tue Nov  3 10:47:19 AM UTC 2026
Écrit sur drbd-aws le 2026-11-03T10:12:38+00:00
test bascule Tue Nov  3 10:47:19 AM UTC 2026
r0 role:Secondary
  disk:UpToDate open:no
  drbd-azure role:Secondary
    peer-disk:UpToDate

[rocky@drbd-aws ~]$
```

Sur Azure : promotion en primaire, montage. Le fichier écrit sur AWS est relu à l'identique, et AWS est devenu secondaire `UpToDate`.

```
[rocky@drbd-azure ~]$ sudo drbdadm primary r0
sudo mkdir -p /srv/drbd && sudo mount /dev/drbd0 /srv/drbd
cat /srv/drbd/hello.txt && sudo drbdadm status r0
Écrit sur drbd-aws le 2026-11-03T10:12:38+00:00
test bascule Tue Nov  3 10:47:19 AM UTC 2026
r0 role:Primary
  disk:UpToDate open:yes
  drbd-aws role:Secondary
    peer-disk:UpToDate

[rocky@drbd-azure ~]$
```

6. Configuration DRBD : IP privée locale, IP publique du pair

Les deux fichiers `/etc/drbd.d/r0.res` diffèrent volontairement. Chaque nœud écoute sur sa propre IP privée (`10.10.1.189` sur AWS, `10.20.1.4` sur Azure) et joint son pair sur l'IP publique de celui-ci, à cause du NAT 1:1 des deux clouds.

```
[rocky@drbd-aws ~]$ cat /etc/drbd.d/r0.res
resource r0 {
  device    /dev/drbd0;
  disk      /dev/vg_drbd/lv_drbd;
  meta-disk internal;

  net {
    protocol    C;       # synchrone : écriture validée sur les 2 nœuds
    verify-alg  sha256;  # permet "drbdadm verify r0"
  }

  on drbd-aws {
    node-id 0;
    address 10.10.1.189:7789;
  }

  on drbd-azure {
    node-id 1;
    address 20.251.28.85:7789;
  }
}
[rocky@drbd-aws ~]$
```

```
[rocky@drbd-azure ~]$ cat /etc/drbd.d/r0.res
resource r0 {
  device    /dev/drbd0;
  disk      /dev/vg_drbd/lv_drbd;
  meta-disk internal;

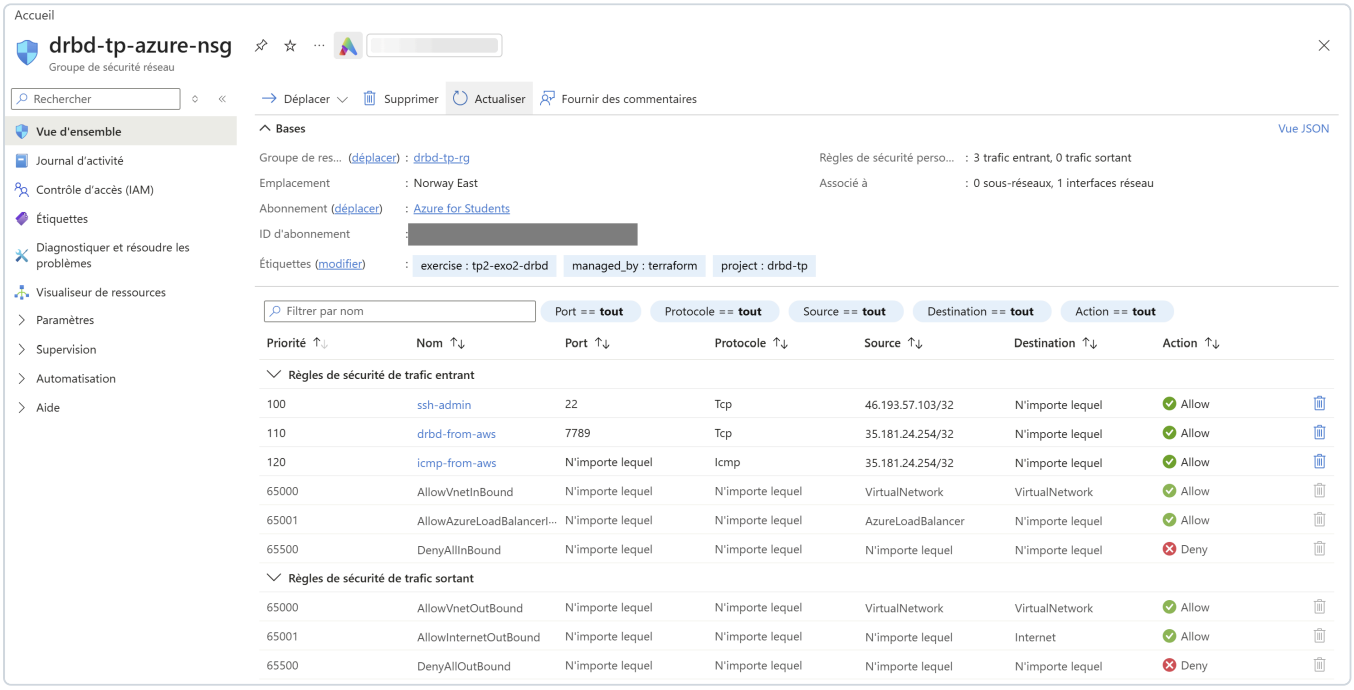
  net {
    protocol    C;       # synchrone : écriture validée sur les 2 nœuds
    verify-alg  sha256;  # permet "drbdadm verify r0"
  }

  on drbd-aws {
    node-id 0;
    address 35.181.24.254:7789;
  }

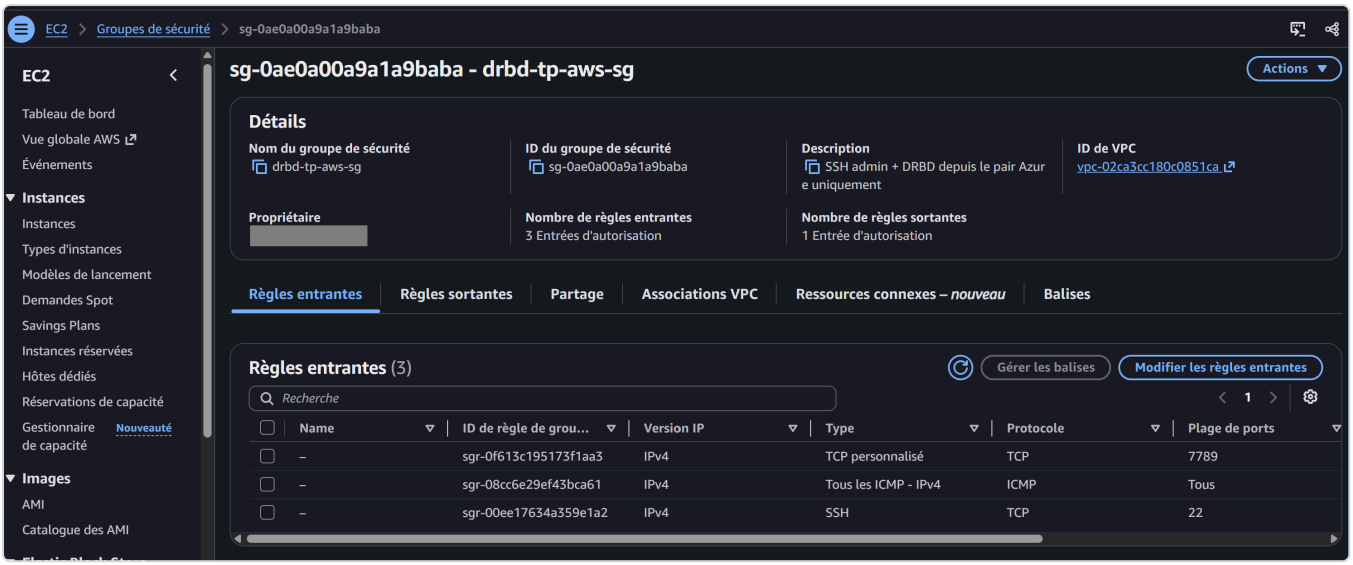
  on drbd-azure {
    node-id 1;
    address 10.20.1.4:7789;
  }
}
[rocky@drbd-azure ~]$
```

7. Filtrage réseau

NSG Azure : SSH ouvert uniquement à l'IP d'administration, port DRBD 7789 et ICMP ouverts uniquement à l'IP publique AWS (`/32`). Tout le reste est refusé (`DenyAllInBound`).



Security group AWS : trois règles entrantes seulement, port 7789, ICMP et SSH 22. Les sources, en `/32`, sont définies dans `aws.tf` : l'IP publique Azure pour 7789 et ICMP, l'IP d'administration pour 22.



Les identifiants de compte AWS et d'abonnement Azure sont masqués sur les captures.

8. Nettoyage

```
Destroy complete! Resources: 24 destroyed.
```

Contrôle après le `destroy` : state Terraform vide. Côté AWS, aucune instance, aucun volume, aucune Elastic IP, aucune NAT Gateway, aucun snapshot, aucun VPC ni clé `drbd-tp`, dans aucune région. Côté Azure, `drbd-tp-rg` a disparu ; il ne reste que `NetworkWatcherRG`, créé automatiquement par Azure et gratuit.

Destruction

```
terraform destroy
```

Limites connues (contexte TP)

- **Trafic DRBD en clair sur Internet.** Le filtrage `/32` limite qui peut se connecter, mais les données ne sont pas chiffrées. En production, faire passer DRBD dans un tunnel WireGuard ou IPsec (VPN Gateway, au prix du coût).
- **Protocole C sur un WAN** (environ 30 ms Paris–Oslo) : chaque écriture attend l'acquittement distant. Le protocole A (asynchrone) serait plus rapide, avec un risque de perte des dernières écritures.
- **Pas de gestionnaire de cluster** (Pacemaker) ni de quorum. La bascule est manuelle et rien n'empêche un split-brain. Avec 2 nœuds, un 3e nœud « diskless tiebreaker » serait nécessaire pour un quorum DRBD.